

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»**

**Навчально-науковий інститут прикладного системного аналізу
Кафедра системного проектування**

Звіт

**про виконання лабораторної роботи №2
з дисципліни «Комп'ютерні мережі»**

Виконав:

студент 3 курсу, групи КН-31

Бенза Р. А.

Тема: АНАЛІЗ МЕРЕЖЕВОГО ТРАФІКУ. АНАЛІЗ РОБОТИ ПРОТОКОЛУ TCP

Мета роботи: Ознайомлення із засобами перехоплення, збереження і аналізу мережевого трафіку за допомогою відомих аналізаторів мережевого трафіку як Wireshark; проведення аналізу мережевого сеансу транспортного протоколу TCP.

Хід роботи

Щоб точно дізнатися IP-адресу і MAC-адресу мережевого адаптера на ПК, відкриємо вікно командного рядка і введемо `ipconfig/all`.

```
Wireless LAN adapter Беспроводная сеть 2:

Connection-specific DNS Suffix . : 
Description . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
Physical Address. . . . . : A0-29-42-8C-2A-03
DHCP Enabled. . . . . : Yes
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . . : fe80::d1be:c439:13ba:fb42%15(Preferred)
IPv4 Address. . . . . : 192.168.1.8(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Lease Obtained. . . . . : 3 березня 2026 р. 11:50:09
Lease Expires . . . . . : 3 березня 2026 р. 15:50:32
Default Gateway . . . . . : 192.168.1.1
DHCP Server . . . . . : 192.168.1.1
DHCPv6 IAID . . . . . : 195045698
DHCPv6 Client DUID. . . . . : 00-01-00-01-2D-48-77-5D-08-BF-B8-09-4A-AD
DNS Servers . . . . . : 192.168.1.1
                        8.8.8.8
                        192.168.1.1
NetBIOS over Tcpip. . . . . : Enabled
```

Рисунок 1 – Специфікації стеку TCP/IP для Wi-Fi інтерфейсу

MAC-адреса (Physical Address): A0-29-42-8C-2A-03

IP-адреса (IPv4 Address): 192.168.1.8

Для початку практичного дослідження в інтерфейсі програми Wireshark було обрано бездротовий адаптер Wi-Fi, специфікації якого ми визначили на попередньому етапі. Після вибору правильного мережевого інтерфейсу, у верхньому меню Capture було активовано команду Start, що запустило процес перехоплення та аналізу всіх мережевих кадрів канального рівня в режимі реального часу. З цього

моменту сніфер почав фіксувати потоковий фоновий трафік і підготувався до реєстрації нашого цільового веб запиту.

Оскільки більшість сучасних веб ресурсів (зокрема й указані в методичних рекомендаціях) наразі примусово використовують шифрування HTTPS, для чистоти експерименту й коректного дослідження незашифрованого протоколу HTTP було обрано альтернативний сайт <http://httpforever.com/>. Цей ресурс ідеально підходить, оскільки містить просту вітальну сторінку без великої кількості картинок чи сторонніх скриптів, що дозволяє зафіксувати чіткий HTTP-обмін без зайвого мережевого шуму.

Одразу після того, як браузер повністю відобразив вміст сторінки, вікно оглядача було згорнуто для повернення до програми Wireshark. Щоб уникнути засмічення дамп-файлу паразитним трафіком від інших фонових додатків чи операційної системи, у меню Capture було відразу обрано команду Stop. У результаті ми успішно сформували й зупинили захоплення пакетів, отримавши локальний зріз мережевої активності, який містить повний цикл взаємодії нашого ПК з віддаленим сервером для подальшого аналізу.

Аналіз статистики

Використовуючи меню Statistics (Capture file properties, Protocol Hierarchy, Packet lengths) визначимо статистичні дані по захопленим пакетам:

1) Процентне співвідношення трафіку різних протоколів в мережі:

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
▼ Frame	100.0	1040	100.0	445321	276 k	0	0	0	1040
▼ Ethernet	100.0	1040	3.3	14626	9096	0	0	0	1040
▼ Internet Protocol Version 6	0.1	1	0.0	40	24	0	0	0	1
▼ User Datagram Protocol	0.1	1	0.0	8	4	0	0	0	1
▼ Multicast Domain Name System	0.1	1	0.0	61	37	1	61	37	1
▼ Internet Protocol Version 4	99.9	1039	4.7	20780	12 k	0	0	0	1039
▼ User Datagram Protocol	62.2	647	1.2	5176	3219	0	0	0	647
▼ Simple Service Discovery Protocol	0.9	9	0.9	3959	2462	9	3959	2462	9
▼ QUIC IETF	43.5	452	65.1	289858	180 k	452	283181	176 k	472
▼ NetBIOS Name Service	0.2	2	0.0	100	62	2	100	62	2
▼ Multicast Domain Name System	0.4	4	0.0	193	120	4	193	120	4
▼ Data	17.3	180	6.4	28287	17 k	180	28287	17 k	180
▼ Transmission Control Protocol	37.7	392	1.9	8524	5301	283	6344	3945	392
▼ Transport Layer Security	5.0	52	9.0	39868	24 k	52	39868	24 k	52
▼ Hypertext Transfer Protocol	1.6	17	3.5	15406	9581	8	3265	2030	17
▼ Media Type	0.2	2	1.2	5296	3293	2	5296	3293	2
▼ Line-based text data	0.3	3	5.2	23097	14 k	3	23097	14 k	3
▼ HTML Form URL Encoded	0.1	1	0.0	144	89	1	144	89	1
▼ eXtensible Markup Language	0.3	3	0.3	1409	876	3	1409	876	3
▼ Domain Name System	3.8	40	0.6	2690	1673	40	2690	1673	40

Рисунок 2 – Процентне співвідношення трафіку різних протоколів в мережі

Аналіз розподілу пакетів за рівнями моделі OSI та стеку TCP/IP показує таку картину:

Канальний рівень:

- Ethernet: Охоплює 100% усього трафіку (1040 пакетів), оскільки є базовою технологією підключення ПК до локальної мережі.

Мережевий рівень:

- Internet Protocol Version 4 (IPv4): 99.9% від усіх пакетів (1039 шт.). Саме через нього здійснювався основний обмін даними та вебсерфінг.
- Internet Protocol Version 6 (IPv6): Складає лише 0.1% (всього 1 пакет), що відображає мінімальну активність операційної системи у цьому просторі адресації.

Транспортний рівень:

- User Datagram Protocol (UDP): Неочікувано займає більшу частину транспортного рівня – 62.2% (647 пакетів). Його домінування зумовлене швидкими фоновими запитами без підтвердження доставки.

- Transmission Control Protocol (TCP): Охоплює 37.7% від усіх пакетів (392 шт.). Він забезпечує надійну доставку даних із встановленням з'єднання, що необхідно для роботи прикладних вебдодатків.

Прикладний рівень:

- QUIC IETF: Складає 43.5% від загальної кількості пакетів, але генерує аж 65.1% від усього об'єму інформації в байтах (452 пакети). Це свідчить про активну роботу важких зашифрованих фонових сесій: сервісів Google у сусідніх вкладках під час тесту.
- Domain Name System (DNS): Займає 3.8% (40 пакетів), відповідаючи за попереднє розв'язання текстових доменних імен у цифрові IP-адреси.
- Hypertext Transfer Protocol (HTTP): Становить 1.6% (всього 17 пакетів). Це наш чистий цільовий обмін із сервером – сюди увійшли безпосередньо текстові дані вебсторінки, медіафайли.
- Transport Layer Security (TLS): Займає 5.0% (52 пакети), забезпечуючи стандартне криптографічне шифрування для інших захищених HTTPS-з'єднань у фоні.

2)Середня швидкість біт / сек, середня швидкість байт / сек:

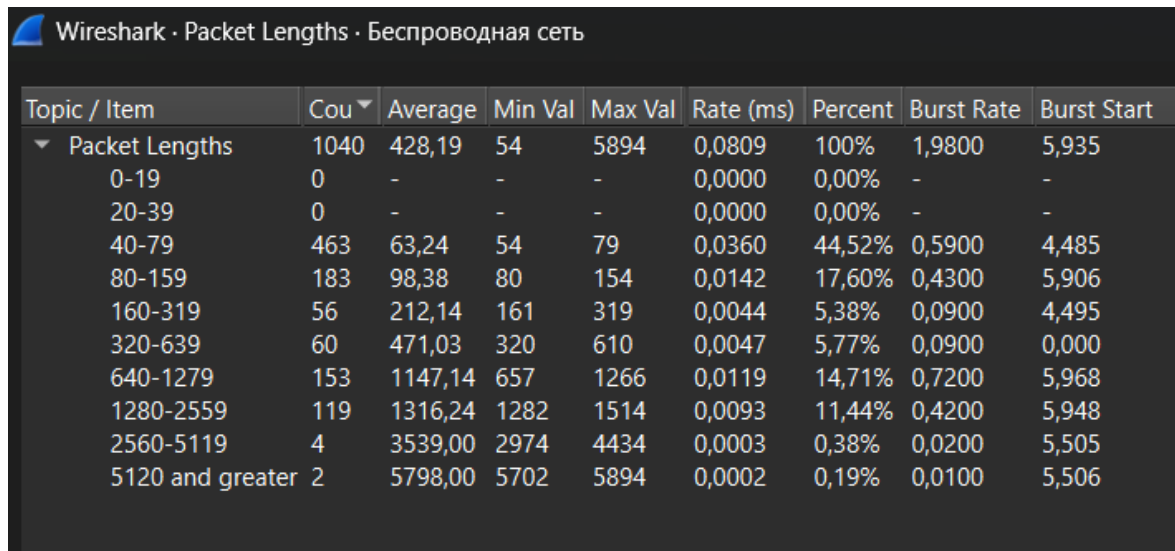
Statistics			
Measurement	Captured	Displayed	Marked
Packets	1040	17 (1.6%)	—
Time span, s	12.863	3.777	—
Average pps	80.9	4.5	—
Average packet size, B	428	749	—
Bytes	445321	12725 (2.9%)	0
Average bytes/s	34 k	3368	—
Average bits/s	276 k	26 k	—

Рисунок 3 – Властивості файлу захоплення

Аналіз вікна статистичних властивостей дозволяє виділити ключові метрики динаміки трафіку в межах нашої сесії. Загальна тривалість перехоплення склала 12.863 секунд, протягом яких сніфер зареєстрував 1040 пакетів сумарним об'ємом 445 321 байт. Середня швидкість передачі даних у мережі зафіксована на рівні 276 кбіт/с (Average bits/s), що в байтовому еквіваленті становить 34 кБайт/с (Average

bytes/s). Також важливим інформаційним параметром є усереднений розмір одного пакета, значення якого склало 428 байтів, що вказує на збалансоване співвідношення між службовими заголовками та корисним навантаженням.

3) Мінімальний, максимальний та середній розміри пакету:



Wireshark · Packet Lengths · Беспроводная сеть

Topic / Item	Cou	Average	Min Val	Max Val	Rate (ms)	Percent	Burst Rate	Burst Start
▼ Packet Lengths	1040	428,19	54	5894	0,0809	100%	1,9800	5,935
0-19	0	-	-	-	0,0000	0,00%	-	-
20-39	0	-	-	-	0,0000	0,00%	-	-
40-79	463	63,24	54	79	0,0360	44,52%	0,5900	4,485
80-159	183	98,38	80	154	0,0142	17,60%	0,4300	5,906
160-319	56	212,14	161	319	0,0044	5,38%	0,0900	4,495
320-639	60	471,03	320	610	0,0047	5,77%	0,0900	0,000
640-1279	153	1147,14	657	1266	0,0119	14,71%	0,7200	5,968
1280-2559	119	1316,24	1282	1514	0,0093	11,44%	0,4200	5,948
2560-5119	4	3539,00	2974	4434	0,0003	0,38%	0,0200	5,505
5120 and greater	2	5798,00	5702	5894	0,0002	0,19%	0,0100	5,506

Рисунок 4 – Статистика розподілу довжини пакетів

Аналіз вікна статистики Packet Lengths показує, що мережа переважно працювала в режимі інтерактивного обміну та підтвердження даних, оскільки більше ніж 62% від усіх зафіксованих кадрів припадає на дрібні пакети розміром від 40 до 159 байт. Мінімальний розмір пакету (54 байти) представляє стандартний службовий мінімум для Ethernet-мереж (TCP ACK-підтвердження або ARP-запити), які не несуть корисного тексту чи медіа-даних, а лише координують з'єднання. Середній розмір (428,19 байт) зміщений ближче до нижньої межі, що ще раз підтверджує домінування коротких команд у сесії над масивним завантаженням важких файлів. Водночас аномальний максимальний розмір пакету (5894 байти) суттєво перевищує стандартний ліміт заліза MTU в 1500 байт; це однозначно вказує на роботу механізмів оптимізації мережевого драйвера операційної системи (технології LRO / TSO), яка програмно зклеює кілька послідовних кадрів, що прилетіли один за одним, в один гігантський пакет задля економії ресурсів центрального процесора перед тим, як передати цей трафік у Wireshark.

4) Ступінь використання смуги пропускання каналу (завантаження мережі):

Середня швидкість передачі даних під час захоплення трафіку становила 276 Кбіт/с. Хоча домашній роутер підтримує швидкість до 1 Гбіт/сек, реальна швидкість мережі обмежена провайдером до 100 Мбіт/сек. Тоді:

$$U = \frac{276\,000}{100\,000\,000} = 0.276\%$$

Отримане значення свідчить про роботу мережевого інтерфейсу в режимі мінімального навантаження, оскільки короточасний НТТР-обмін текстовими даними разом з фоновими мережевими процесами фактично не створює трафікового тиску на смугу пропускання.

5) EndPoints:

Ethernet · 13	IPv4 · 39	IPv6 · 5	TCP · 74	UDP · 28		
Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes
00:45:e2:a8:48:25	990	525 kB	567	373 kB	423	151 kB
01:00:5e:00:00:02	3	138 bytes	0	0 bytes	3	138 bytes
01:00:5e:00:00:16	15	810 bytes	0	0 bytes	15	810 bytes
01:00:5e:00:00:fb	16	1 kB	0	0 bytes	16	1 kB
01:00:5e:00:00:fc	1	50 bytes	0	0 bytes	1	50 bytes
01:00:5e:7f:ff:fa	11	4 kB	0	0 bytes	11	4 kB
0a:0f:7c:6f:32:23	23	2 kB	23	2 kB	0	0 bytes
33:33:00:00:00:02	1	70 bytes	0	0 bytes	1	70 bytes
33:33:00:00:00:16	3	270 bytes	0	0 bytes	3	270 bytes
33:33:00:00:00:fb	7	840 bytes	0	0 bytes	7	840 bytes
5c:e9:31:35:0c:28	1,013	530 kB	455	157 kB	558	373 kB
f2:1f:9c:fa:cb:af	7	394 bytes	7	394 bytes	0	0 bytes
ff:ff:ff:ff:ff:ff	14	2 kB	0	0 bytes	14	2 kB

Рисунок 5 – EndPoints для Ethernet

Вікно Endpoints відображає всі кінцеві вузли, які брали участь у мережевому обміні під час трасування.

На вкладці Ethernet зафіксовано 13 унікальних фізичних пристроїв, серед яких основними є такі:

- 00:45:e2:a8:48:25 – це власна MAC-адреса нашого бездротового адаптера ПК (зафіксовано 990 пакетів), яка виступала ініціатором більшості запитів (Tx: 567 пакетів) та отримувала відповіді (Rx: 423 пакети).
- 5c:e9:31:35:0c:28 – MAC-адреса нашого основного шлюзу локальної мережі (зафіксовано 1013 пакетів), який забезпечував ретрансляцію кадрів і через який здійснювався весь вихід на зовнішні інтернет-ресурси.
- ff:ff:ff:ff:ff:ff – стандартна широкомовна адреса, на яку відправлялися службові кадри (наприклад, пошукові запити протоколу ARP) для опитування всіх вузлів у поточному мережевому сегменті.
- Група адрес, що починаються на 01:00:5e: та 33:33:, представляють собою мультикаст-адреси для розсилання службових повідомлень обмеженим групам пристроїв.

Вкладка IPv4 містить 39 унікальних логічних адрес. Головний обмін даними у цьому вікні відбувається між нашою локальною IP-адресою (192.168.0.156) та мережевою IP-адресою віддаленого вебсервера сайту, до якого ми зверталися через браузер, що підтверджує успішну інкапсуляцію логічних IP-пакетів у фізичні кадри Ethernet між ПК та маршрутизатором.

Ethernet · 13	IPv4 · 39	IPv6 · 5	TCP · 74	UDP · 28								
Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City	Latitude	Longitude	AS Number	AS Organization
8.8.4.4	20	7 kB	11	5 kB	9	2 kB						
9.141.2.80	6	502 bytes	2	186 bytes	4	316 bytes						
13.107.137.11	126	108 kB	57	15 kB	69	94 kB						
20.33.3.2	2	108 bytes	1	54 bytes	1	54 bytes						
20.50.201.200	23	14 kB	12	10 kB	11	4 kB						
20.67.204.227	30	14 kB	15	11 kB	15	3 kB						
34.98.64.218	10	650 bytes	5	355 bytes	5	295 bytes						
34.149.50.64	12	771 bytes	6	421 bytes	6	350 bytes						
35.195.2.28	2	109 bytes	1	54 bytes	1	55 bytes						
40.126.53.16	1	54 bytes	1	54 bytes	0	0 bytes						
50.85.242.216	28	15 kB	14	11 kB	14	3 kB						
52.106.137.11	47	39 kB	27	29 kB	20	10 kB						
52.123.128.14	3	175 bytes	2	120 bytes	1	55 bytes						
64.227.36.222	12	864 bytes	8	532 bytes	4	332 bytes						
74.242.255.116	4	242 bytes	2	132 bytes	2	110 bytes						
95.101.116.81	127	68 kB	62	31 kB	65	37 kB						
98.66.133.186	12	2 kB	6	1 kB	6	863 bytes						
142.250.130.95	20	2 kB	10	2 kB	10	730 bytes						
142.250.147.188	8	484 bytes	4	264 bytes	4	220 bytes						
142.251.98.101	40	22 kB	21	9 kB	19	13 kB						
142.251.98.132	4	242 bytes	2	132 bytes	2	110 bytes						
142.251.151.119	9	2 kB	5	1 kB	4	728 bytes						
149.154.167.41	342	200 kB	107	12 kB	235	188 kB						

Рисунок 6 – EndPoints для IPv4

На вкладці IPv4 вікна Endpoints відображено всі віддалені логічні адреси інтернет-ресурсів, з якими комп'ютер взаємодіяв під час аналізу трафіку. Найбільша мережева активність тут зафіксована з вузлом 149.154.167.41, що належить серверам

месенджера Telegram. Ряд адрес типу 142.250.x.x і 142.251.x.x разом із публічним сервером 8.8.4.4 відображає автоматичну роботу сервісів Google та виконання DNS-запитів для розв'язання доменних імен у браузері. Інші вузли, як-от 13.107.137.11 та 95.101.116.81, представляють розподілену інфраструктуру Microsoft та мережі доставки контенту, які забезпечують життєдіяльність фонових процесів операційної системи. Усі ці зовнішні потоки даних інкапсулюються в кадри Ethernet і проходять через наш локальний шлюз, демонструючи роботу маршрутизації на мережевому рівні.

б) IO Graph поведінка мережевого інтерфейсу.



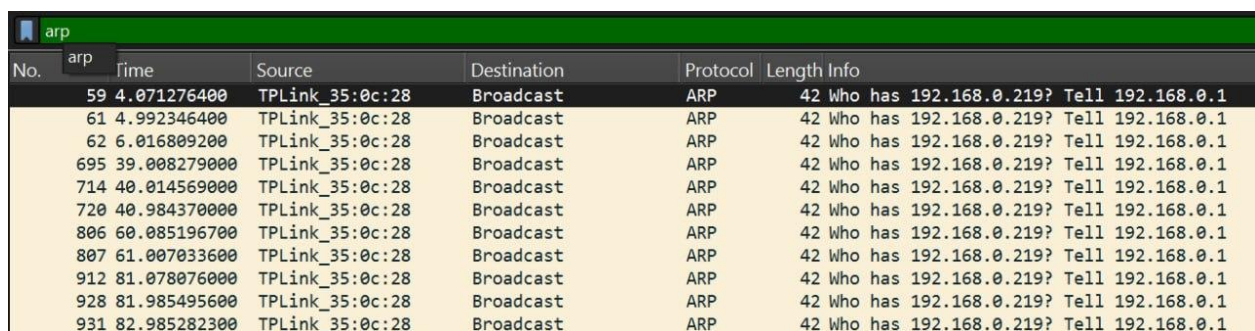
Рисунок 7 – IO Graph мережевого інтерфейсу.

Графік I/O Graphs наочно демонструє динаміку роботи мережевого інтерфейсу, де перші 2.5 секунди спостерігається спокійний фоновий трафік, який у момент завантаження сайту стрімко зростає і сягає свого піку на 5-й секунді – понад 400 пакетів/сек. Цей короточасний сплеск зумовлений активацією фонових

зашифрованих потоків (QUIC/UDP) та супутніх системних запитів, тоді як лінія відфільтрованого HTTP-трафіку залишається практично плоскою біля нуля, підтверджуючи мінімальну вагу самої текстової вебсторінки. Після 6-ї секунди інтенсивність обміну даними так само різко падає, повертаючи інтерфейс у вихідний стан очікування. При цьому повна відсутність помилок у зоні TCP Errors свідчить про високу стабільність і відмінну якість поточного бездротового з'єднання без жодних втрат чи повторних надсилань кадрів.

Аналіз ARP запитів

Наступним кроком подивимося на ARP-запити у Wire-Shark. Для цього від'єднаємося від мережі та зробимо нове прослуховування, під час якого доєднаємося до Інтернету.



No.	Time	Source	Destination	Protocol	Length	Info
59	4.071276400	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
61	4.992346400	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
62	6.016809200	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
695	39.008279000	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
714	40.014569000	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
720	40.984370000	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
806	60.085196700	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
807	61.007033600	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
912	81.078076000	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
928	81.985495600	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1
931	82.985282300	TPLink_35:0c:28	Broadcast	ARP	42	Who has 192.168.0.219? Tell 192.168.0.1

Рисунок 8 – ARP запити

На наведеному рисунку зафіксовано серію службових широкомовних запитів протоколу ARP, який відповідає за динамічне відображення мережевих IP-адрес у фізичні MAC-адреси пристроїв у локальному сегменті. Оскільки відправник знає лише IP-адресу цільового хоста, але не знає його MAC-адреси для формування кадру канального рівня, він надсилає запит на адресу призначення Broadcast (широкомовна MAC-адреса ff:ff:ff:ff:ff:ff). Це змушує абсолютно всі пристрої в поточному локальному підмережевому просторі прийняти й прочитати цей пакет. Мережевий вузол із логічною адресою 192.168.0.1 послідовно опитує оточення командою. Тобто він намагається з'ясувати фізичну адресу пристрою, якому наразі виділено конкретну IP-адресу, щоб мати змогу передати йому дані безпосередньо. Усі пакети мають стандартний мінімальний розмір для ARP-запитів – 42 байти.

IP-адреса шлюзу: 192.168.0.1

MAC-адреса шлюзу: 5c:e9:31:35:0c:28 (у колонці Source програма Wireshark для зручності замінила перші три байти OUI-ідентифікатора назвою виробника мережевого заліза, відобразивши адресу як TPLink_35:0c:28).

Аналіз DNS пакетів

Далі дослідимо DNS пакети.

No.	Time	Source	Destination	Protocol	Length	Info
145	3.808406400	192.168.0.111	192.168.0.1	DNS	82	Standard query 0x0528 HTTPS google.com
149	3.812555300	192.168.0.1	192.168.0.111	DNS	180	Standard query response 0x5191 A google.com A 142.250.130.113 A 142.250.130.139 A 142.250.130.116
151	3.812853800	192.168.0.1	192.168.0.111	DNS	109	Standard query response 0x0528 HTTPS google.com HTTPS
193	4.488565600	192.168.0.111	192.168.0.1	DNS	107	Standard query 0xb88 HTTPS optimizationguide-pa.googleapis.com
197	4.489486800	192.168.0.111	192.168.0.1	DNS	107	Standard query 0x7e25 A optimizationguide-pa.googleapis.com
202	4.495165800	192.168.0.1	192.168.0.111	DNS	166	Standard query response 0xb88 HTTPS optimizationguide-pa.googleapis.com SOA ns1.google.com
204	4.495673100	192.168.0.1	192.168.0.111	DNS	173	Standard query response 0x7e25 A optimizationguide-pa.googleapis.com A 216.239.38.223 A 216.239.38.223
227	4.526300400	192.168.0.111	192.168.0.1	DNS	87	Standard query 0x796 HTTPS httpforever.com
231	4.527507400	192.168.0.111	192.168.0.1	DNS	87	Standard query 0xdd16 A httpforever.com
235	4.528800300	192.168.0.111	192.168.0.1	DNS	106	Standard query 0xec7d HTTPS oauthaccountmanager.googleapis.com
239	4.530544500	192.168.0.111	192.168.0.1	DNS	106	Standard query 0x0955 A oauthaccountmanager.googleapis.com
247	4.533867900	192.168.0.1	192.168.0.111	DNS	165	Standard query response 0xec7d HTTPS oauthaccountmanager.googleapis.com SOA ns1.google.com
251	4.535130300	192.168.0.1	192.168.0.111	DNS	172	Standard query response 0x0955 A oauthaccountmanager.googleapis.com A 216.239.34.223 A 216.239.34.223
298	4.591255700	192.168.0.1	192.168.0.111	DNS	150	Standard query response 0x796 HTTPS httpforever.com SOA karina.ns.cloudflare.com
300	4.593052800	192.168.0.1	192.168.0.111	DNS	105	Standard query response 0xdd16 A httpforever.com A 146.190.62.39
348	4.770636500	192.168.0.111	192.168.0.1	DNS	95	Standard query 0xf94c HTTPS safebrowsing.google.com
351	4.771522800	192.168.0.111	192.168.0.1	DNS	95	Standard query 0xa919 A safebrowsing.google.com
354	4.774105500	192.168.0.1	192.168.0.111	DNS	166	Standard query response 0xf94c HTTPS safebrowsing.google.com CNAME sb.l.google.com SOA ns1.google.com
359	4.775905700	192.168.0.111	192.168.0.1	DNS	132	Standard query response 0xa919 A safebrowsing.google.com CNAME sb.l.google.com A 142.251.37.110
414	5.106089100	192.168.0.111	192.168.0.1	DNS	92	Standard query 0xcac4 HTTPS cdnjs.cloudflare.com
418	5.106893700	192.168.0.111	192.168.0.1	DNS	92	Standard query 0xfeed A cdnjs.cloudflare.com
422	5.111386100	192.168.0.1	192.168.0.111	DNS	167	Standard query response 0xcac4 HTTPS cdnjs.cloudflare.com HTTPS
425	5.111386100	192.168.0.1	192.168.0.111	DNS	126	Standard query response 0xfeed A cdnjs.cloudflare.com A 104.17.24.14 A 104.17.25.14
555	5.736246900	192.168.0.111	192.168.0.1	DNS	83	Standard query 0xedf3 HTTPS s.deepl.com
558	5.736495800	192.168.0.111	192.168.0.1	DNS	83	Standard query 0xd25a A s.deepl.com
562	5.737774800	192.168.0.111	192.168.0.1	DNS	92	Standard query 0xc44e HTTPS fonts.googleapis.com
566	5.738421300	192.168.0.111	192.168.0.1	DNS	92	Standard query 0x172e A fonts.googleapis.com
571	5.742877800	192.168.0.1	192.168.0.111	DNS	120	Standard query response 0xd25a A s.deepl.com CNAME s-lb.deepl.com A 194.124.205.65
575	5.743410600	192.168.0.1	192.168.0.111	DNS	151	Standard query response 0xc44e HTTPS fonts.googleapis.com SOA ns1.google.com
578	5.743410600	192.168.0.1	192.168.0.111	DNS	110	Standard query response 0x172e A fonts.googleapis.com A 216.58.207.10
600	5.763076900	192.168.0.1	192.168.0.111	DNS	167	Standard query response 0xedf3 HTTPS s.deepl.com CNAME s-lb.deepl.com SOA michelle.ns.cloudflare.com
669	5.939272800	192.168.0.111	192.168.0.1	DNS	89	Standard query 0x4015 HTTPS fonts.gstatic.com
673	5.940933800	192.168.0.111	192.168.0.1	DNS	89	Standard query 0xe98b A fonts.gstatic.com
676	5.942785100	192.168.0.1	192.168.0.111	DNS	116	Standard query response 0x4015 HTTPS fonts.gstatic.com HTTPS

Рисунок 9 – DNS запити

На наведеному скріншоті відображено роботу служби доменних імен (DNS), яка трансформує текстові адреси сайтів у числові IP-адреси. Мій ПК з IP-адресою надсилає запити типу «Standard query A», а сервер повертає відповіді «Standard query response», що містять цільові IPv4-адреси. Виділений рядок під номером 300 є пакетом-відповіддю від DNS-сервера сайту. У ньому вказано, що для доменного імені httpforever.com успішно знайдено відповідний A-запис (IPv4-адресу) – 146.190.62.39.

IP-адреса DNS-сервера: 192.168.0.1.

IP-адреса веб-сервера сайту: 146.190.62.39

Розрахунок часу відгуку

Для розрахунку часу відгуку подивимося на всі HTTP-запити перехоплені під час прослуховування.

No.	Time	Source	Destination	Protocol	Length	Info
367	4.779230200	192.168.0.111	146.190.62.39	HTTP	514	GET / HTTP/1.1
395	4.960118900	146.190.62.39	192.168.0.111	HTTP	1350	HTTP/1.1 200 OK (text/html)
419	5.107490600	192.168.0.111	146.190.62.39	HTTP	400	GET /js/init.min.js HTTP/1.1
523	5.294180000	146.190.62.39	192.168.0.111	HTTP	440	HTTP/1.1 200 OK (application/javascript)
527	5.324566400	192.168.0.111	146.190.62.39	HTTP	418	GET /css/style.min.css HTTP/1.1
528	5.325035500	192.168.0.111	146.190.62.39	HTTP	423	GET /css/style-wide.min.css HTTP/1.1
537	5.506539700	146.190.62.39	192.168.0.111	HTTP	972	HTTP/1.1 200 OK (text/css)
541	5.685149200	146.190.62.39	192.168.0.111	HTTP	1232	HTTP/1.1 200 OK (text/css)
643	5.881459900	192.168.0.111	146.190.62.39	HTTP	485	GET /css/images/banner.svg HTTP/1.1
644	5.882685500	192.168.0.111	146.190.62.39	HTTP	500	GET /css/images/header-major-on-light.svg HTTP/1.1
881	6.059947700	192.168.0.111	146.190.62.39	HTTP	458	GET /favicon.ico HTTP/1.1
882	6.061501300	146.190.62.39	192.168.0.111	HTTP/X...	1381	HTTP/1.1 200 OK
883	6.062445800	146.190.62.39	192.168.0.111	HTTP/X...	1331	HTTP/1.1 200 OK
884	6.065421100	192.168.0.111	146.190.62.39	HTTP	499	GET /css/images/header-major-on-dark.svg HTTP/1.1
893	6.236139100	146.190.62.39	192.168.0.111	HTTP	787	HTTP/1.1 200 OK (image/x-icon)
895	6.244534300	146.190.62.39	192.168.0.111	HTTP/X...	1337	HTTP/1.1 200 OK
988	8.556444100	192.168.0.111	149.154.167.41	HTTP	198	POST /api HTTP/1.1 (application/x-www-form-urlencoded)

Рисунок 10 – HTTP запити

Для визначення інтервалу часу між запитом та відповіддю скористаємося даними перших двох пакетів цільової HTTP-сесії на скріншоті:

- Час відправки запиту GET (пакет № 367): 4.779 с
- Час отримання відповіді 200 OK (пакет № 395): 4.960
- Розрахунок: $4.960 - 4.779 = 0.181$ секунд (або приблизно 181 мс).

Отриманий показник свідчить про швидку реакцію сервера на запити користувача та швидку обробку.

Дослідження заголовків у HTTP повідомленні

Оберемо повідомлення GET протоколу HTTP та проаналізуємо його вміст.

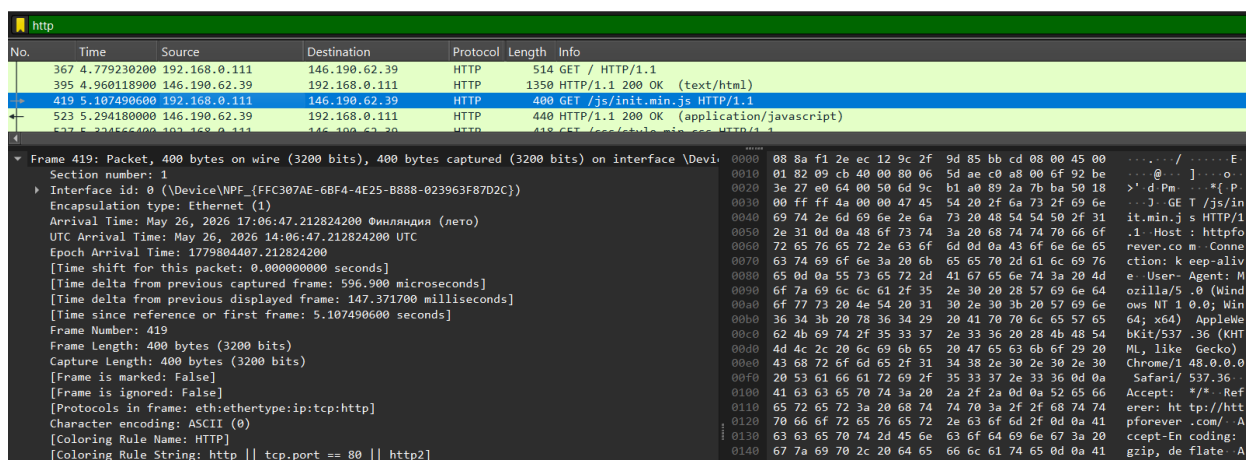


Рисунок 11 – Обране повідомлення HTTP

На малюнку наведені метадані самого аналізатора Wireshark, який розписує фізичні параметри перехоплення. Найголовніші рядки з цього списку та їхнє призначення:

- **Frame Length / Capture Length (400 bytes):** Фізичний розмір кадра на каналному рівні. Це означає, що весь пакет разом із абсолютно всіма службовими заголовками та корисними даними займає в мережі рівно 400 байтів.
- **Arrival Time:** Точний час, коли мережевий адаптер перехопив цей пакет.
- **Frame Number (419):** Порядковий номер цього пакета у поточному файлі захоплення.
- **[Protocols in frame: eth:ethertype:ip:tcp:http]:** Повний список протоколів, які Wireshark зміг розпізнати всередині цього кадра, починаючи від найнижчого каналного рівня і закінчуючи прикладним.

Далі проаналізуємо призначення основних полів Ethernet кадру.

```
▼ Ethernet II, Src: CyberTANTech_a8:48:25 (00:45:e2:a8:48:25), Dst: TPLink_35:0c:28 (5c:e9:31:35:0c:28)
  ▼ Destination: TPLink_35:0c:28 (5c:e9:31:35:0c:28)
    .... ..0. .... = LG bit: Globally unique address (factory default)
    .... ..0. .... = IG bit: Individual address (unicast)
  ▶ Source: CyberTANTech_a8:48:25 (00:45:e2:a8:48:25)
    Type: IPv4 (0x0800)
    [Stream index: 0]
```

Рисунок 12 - Поля заголовка кадру протоколу Ethernet

Опис полів заголовка Ethernet II:

- **Destination (5c:e9:31:35:0c:28):** Фізична MAC-адреса пристрою-отримувача в локальній мережі. Тут зафіксовано адресу шлюзу (маршрутизатора TP-Link), якому комп'ютер пересилає цей кадр для подальшого виходу в Інтернет.
- **Source (00:45:e2:a8:48:25):** Фізична MAC-адреса пристрою-відправника. Це апаратна адреса бездротового Wi-Fi адаптера власного ПК.
- **Type (IPv4 (0x0800)):** Поле типу протоколу вищого рівня. Шістнадцятковий код 0x0800 сповіщає, що всередину кадра інкапсульовано пакет протоколу IPv4, якому і передаються дані для обробки на мережевому рівні.

Наступним кроком проведемо аналіз заголовків IP дейтаграми.

```
▼ Internet Protocol Version 4, Src: 192.168.0.111, Dst: 146.190.62.39
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 386
    Identification: 0x09cb (2507)
  ▶ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x5dae [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.0.111
    Destination Address: 146.190.62.39
    [Stream index: 15]
```

Рисунок 13 – Поля заголовка IP-дейтаграми

На наведеному рисунку розкрито структуру пакета мережевого рівня. Призначення його ключових полів:

- **Version (4):** Визначає версію інтернет-протоколу. Значення 4 вказує на використання стандарту IPv4.
- **Header Length (20 bytes):** Показує довжину самого заголовка пакета. Значення 20 байтів свідчить про те, що заголовок є стандартним і не містить додаткових необов'язкових полів.
- **Total Length (386):** Загальна довжина IP-пакета у байтах, яка враховує як сам IP-заголовок, так і всі вкладені дані транспортного й прикладного рівнів всередині нього.
- **Identification (0x09cb):** Унікальний ідентифікатор пакета, який використовується для правильної збірки початкової дейтаграми, якщо на маршруті відбудеться її фрагментація
- **Flags (0x2, Don't fragment):** Керуючі прапорці фрагментації. Код 0x2 забороняє проміжним маршрутизаторам ділити цей пакет на частини, навіть якщо його розмір перевищує MTU каналу.
- **Fragment Offset (0):** Зсув фрагмента. Оскільки пакет не фрагментований, значення дорівнює нулю.
- **Time to Live (128):** Час життя пакета (TTL), що визначає максимальну кількість проміжних маршрутизаторів, яку він може пройти. Кожен хоп зменшує це число на одиницю для запобігання нескінченному зацикленню пакета в мережі.
- **Protocol (TCP (6)):** Вказує код протоколу наступного (транспортного) рівня, якому потрібно передати корисний вміст дейтаграми після зняття IP-заголовка. Код 6 відповідає протоколу TCP.
- **Header Checksum (0x5dae):** Контрольна сума для перевірки цілісності та коректності даних самого заголовка IP.
- **Source Address (192.168.0.111):** Логічна мережева IP-адреса комп'ютера-відправника в локальній мережі.

- **Destination Address (146.190.62.39):** Публічна мережева IP-адреса віддаленого вебсервера призначення, де розміщено цільовий сайт.

Далі проведемо аналіз заголовків TCP сегменту:

```

Transmission Control Protocol, Src Port: 57444, Dst Port: 80, Seq: 461, Ack: 2757, Len: 346
  Source Port: 57444
  Destination Port: 80
  [Stream index: 11]
  [Stream Packet Number: 9]
  ▶ [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 346]
  Sequence Number: 461      (relative sequence number)
  Sequence Number (raw): 1838985632
  [Next Sequence Number: 807      (relative sequence number)]
  Acknowledgment Number: 2757      (relative ack number)
  Acknowledgment number (raw): 2301262778
  0101 .... = Header Length: 20 bytes (5)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 255
  [Calculated window size: 65280]
  [Window size scaling factor: 256]
  Checksum: 0xff4a [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
  [Client Contiguous Streams: 1]
  [Server Contiguous Streams: 1]
  TCP payload (346 bytes)

```

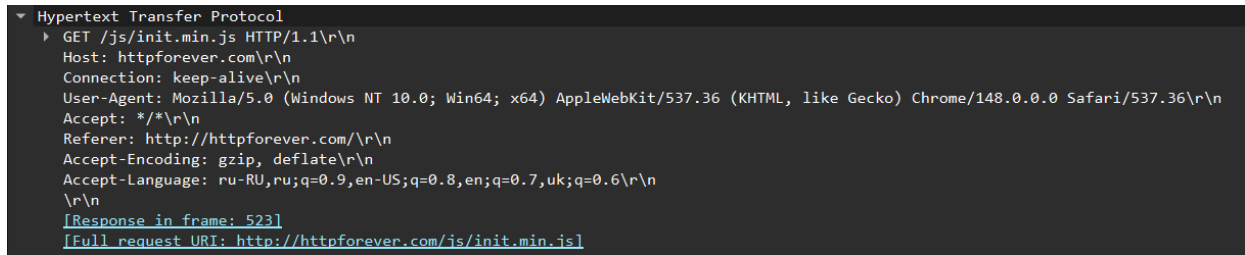
Рисунок 14 – Поля заголовка TCP-сегмента

Призначення основних полів:

- **Source Port (57444):** Номер порту відправника, який операційна система динамічно виділила для конкретної вкладки браузера на твоєму комп'ютері.
- **Destination Port (80):** Номер порту отримувача, що відповідає стандартному порту вебсерверів для обробки незашифрованого HTTP-трафіку.
- **Sequence Number (461):** Відносний порядковий номер першого байта даних цього сегмента в загальному потоці передачі. Він допомагає стороні-отримувачу зібрати пакети в правильному порядку.
- **Next Sequence Number (807):** Очікуваний порядковий номер першого байта наступного сегмента. Обчислюється як сума поточного Sequence Number (461) та довжини корисних даних TCP Segment Len (346).

- **Acknowledgment Number (2757):** Відносний номер підтвердження, який вказує, що всі попередні байти (до номера 2756 включно) від співрозмовника успішно отримані, і ПК очікує від нього байт під номером 2757.
- **Header Length (20 bytes):** Довжина заголовка TCP-сегмента. Значення 20 байтів показує, що заголовок має мінімальний базовий розмір і не містить додаткових опцій.
- **Flags (0x018 (PSH, ACK)):** Керуючі прапорці, що визначають тип і призначення сегмента: ACK – підтверджує легітимність номера у полі Acknowledgment Number. PSH – наказує операційній системі негайно передати ці дані прикладній програмі без накопичення їх у буфері.
- **Window (255):** Поточний розмір вікна одержувача. Він визначає кількість байтів, яку комп'ютер готовий прийняти наразі без отримання додаткового підтвердження.
- **Calculated window size (65280):** Реальний розмір вікна з урахуванням масштабного коефіцієнта, тобто $255 * 256 = 65280$ байтів.
- **Checksum (0xff4a):** Контрольна сума, що використовується для перевірки цілісності та виявлення помилок у заголовку та корисних даних TCP-сегмента.

На рисунку 15 показано вміст полів протоколу HTTP:



```
▼ Hypertext Transfer Protocol
  ▶ GET /js/init.min.js HTTP/1.1\r\n
    Host: httpforever.com\r\n
    Connection: keep-alive\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36\r\n
    Accept: */*\r\n
    Referer: http://httpforever.com/\r\n
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7,uk;q=0.6\r\n
    \r\n
    [Response in frame: 523]
    [Full request URI: http://httpforever.com/js/init.min.js]
```

Рисунок 15 – Структура полів запиту протоколу HTTP

Призначення основних полів HTTP повідомлення:

- **GET /js/init.min.js HTTP/1.1:** Визначає метод запиту, цільовий ресурс та версію протоколу. Метод GET вказує на намір завантажити файл скриптів /js/init.min.js, а HTTP/1.1 задає правила поточної взаємодії.
- **Host: httpforever.com:** Заголовок прикладного рівня, який вказує доменне ім'я сайту. Це необхідно, якщо на одній фізичній IP-адресі сервера розгорнуто кілька різних вебсайтів.
- **Connection: keep-alive:** Керує режимом роботи транспортного TCP-з'єднання. Параметр keep-alive просить сервер не закривати поточну TCP-сесію після відправки цього файлу, щоб через неї можна було завантажити інші елементи сторінки без повторного тристороннього рукошлякування.
- **User-Agent: Mozilla/5.0... Chrome/148.0.0.0...:** Передає серверу детальну інформацію про клієнтське програмне забезпечення. Тут зафіксовано тип операційної системи та версію браузера, що дозволяє серверу адаптувати контент під конкретний пристрій.
- **Accept: */*:** Визначає типи медіаконтенту, які браузер готовий обробити у відповідь. Значення */* означає, що клієнт прийме будь-який формат даних від сервера.
- **Referer: http://httpforever.com/:** Вказує адресу початкової вебсторінки, з якої було здійснено перехід або автоматичний запит до поточного файлу скриптів init.min.js.

- **Accept-Encoding: gzip, deflate:** Повідомляє вебсервер про підтримувані браузером алгоритми стиснення даних для зменшення об'єму трафіку при передачі сторінки по мережі.

Аналіз повної сесії HTTP

Для аналізу повної сесії HTTP з'єднання подивимося на графік потоку.

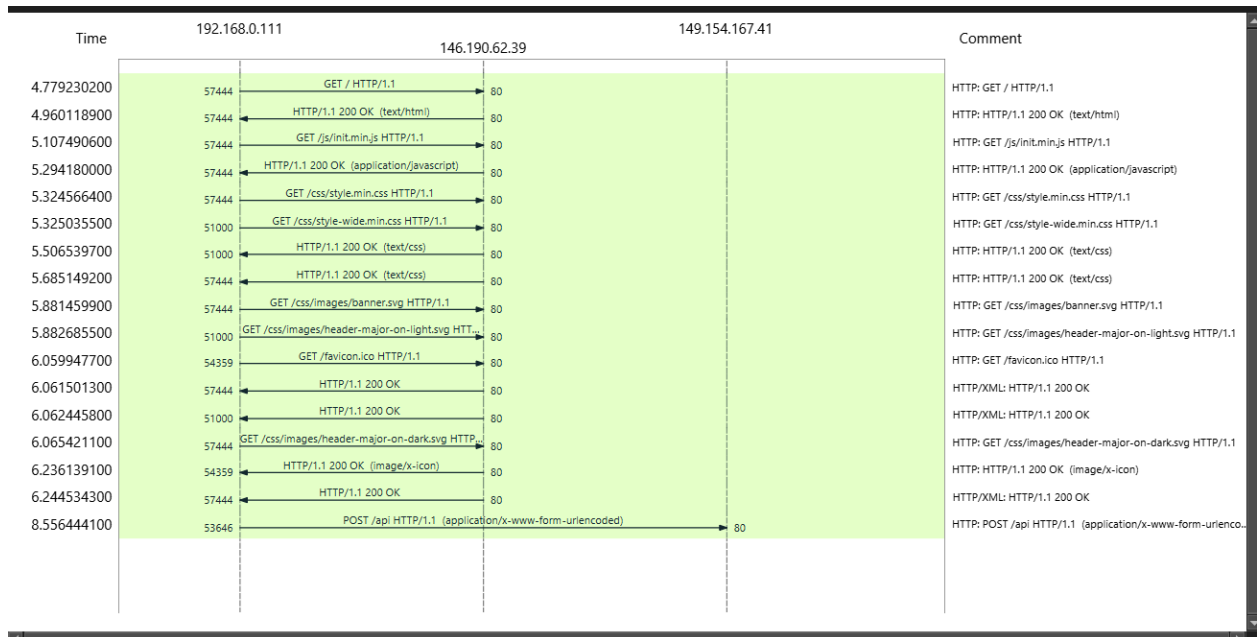


Рисунок 16 – Графік потоку HTTP-запитів

Цей граф потоку відображає хронологію взаємодії між клієнтом та вебсервером на транспортному та прикладному рівнях:

- **Розпаралелювання TCP-потоків:** Для прискорення завантаження сайту браузер одночасно відкриває три паралельні TCP-з'єднання через різні вихідні порти. Основний канал (порт 57444) працює в режимі keep-alive, послідовно викачуючи HTML, скрипти, базові стилі та важку графіку. Допоміжний канал (порт 51000) активується паралельно для одночасного підвантаження додаткових CSS-стилів та SVG-елементів, а порт 54359 виділяється суто під завантаження іконки сайту favicon.ico.
- **Динаміка HTTP-сесії:** Обмін чітко розділений на два етапи. Спочатку клієнт надсилає запит GET/ і за 181 мс отримує базову сторінку 200 OK. Організувавши структуру сторінки, браузер автоматично ініціює серію

повторних GET-запитів для скачування службових файлів, на які сервер успішно повертає статуси 200 OK.

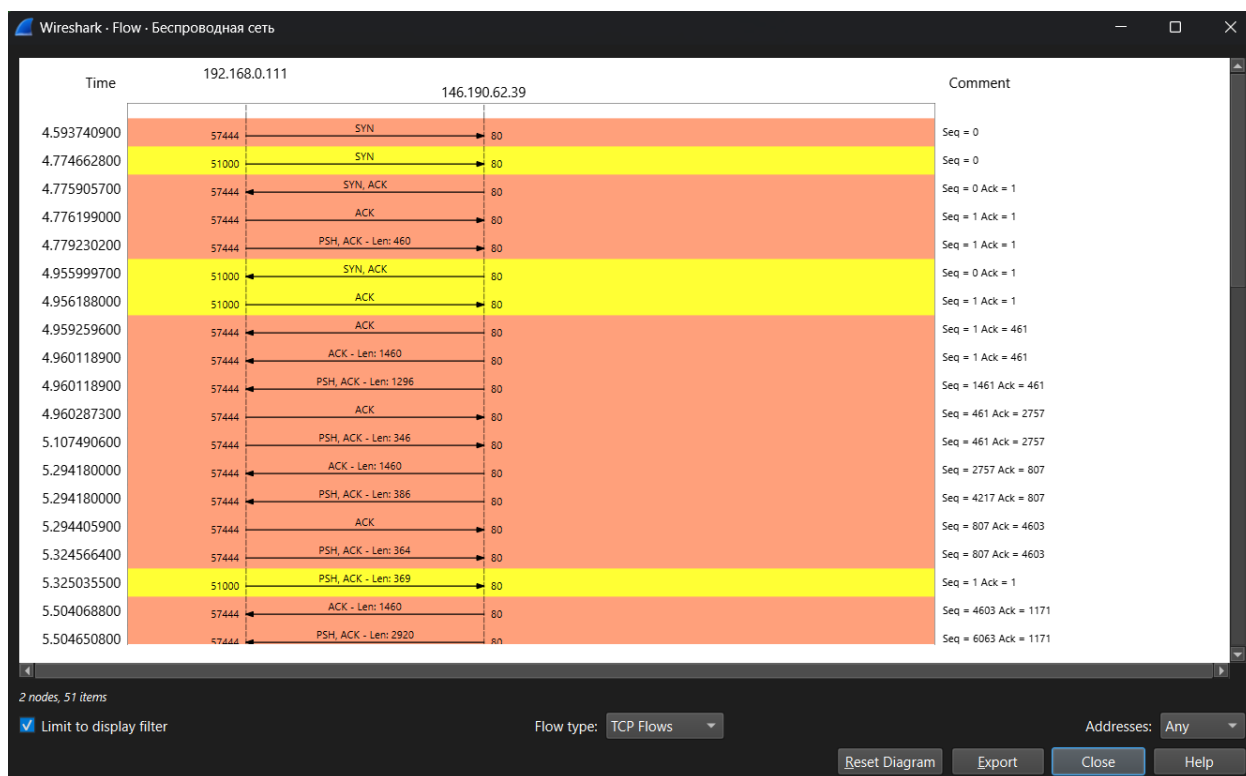


Рисунок 17 – Графік потоку тільки для TCP протоколу

IP-адреси джерела і призначення, номери портів джерела і призначення:

Утилізація мережевого каналу відбувається через три паралельні логічні з'єднання. Усі вони спрямовані на загальну IP-адресу призначення 146.190.62.39 та Dst Port 80 із джерела з IP-адресою 192.168.0.111, але ініціюються з різних портів:

- Перший потік: Src Port: 57444 – основний тривалий канал обміну.
- Другий потік: Src Port: 51000 – допоміжний паралельний канал.
- Третій потік: Src Port: 54359 – виділений канал для швидкого скачування іконок.

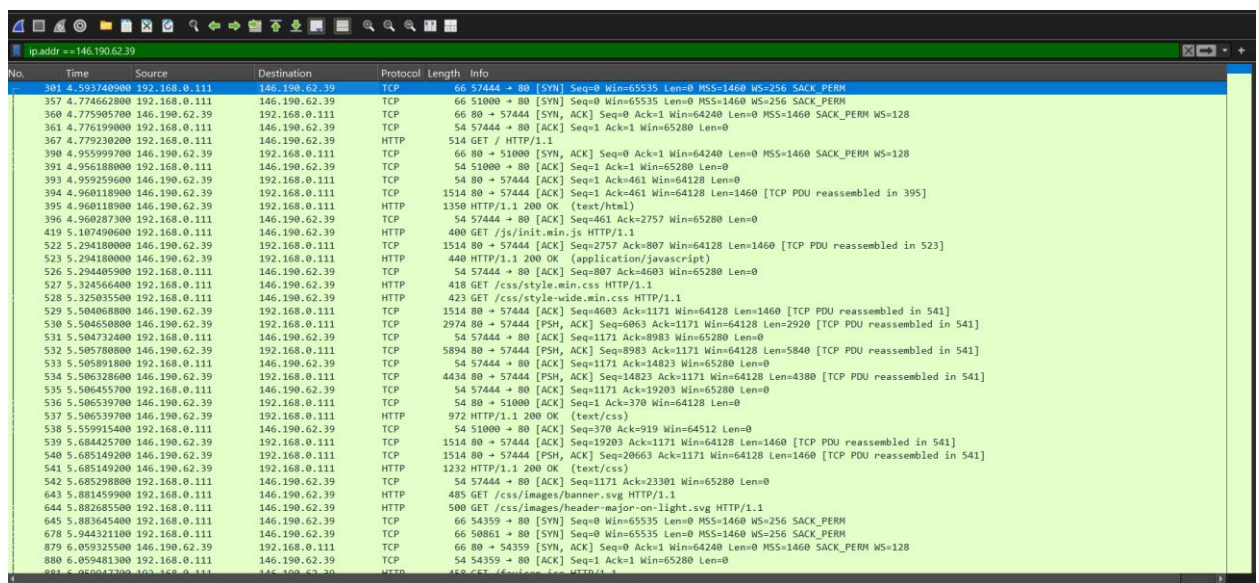
Процедура встановлення з'єднання, передачі даних і розриву з'єднання:

Оскільки ми маємо три порти, які одночасно спілкуються із сервером, опишемо процес лише для основного потоку, для інших етапи є аналогічними. Процес трьохстороннього рукостискання включає такі кроки:

- Основний канал (Порт 57444): Процедура стартує, коли клієнт відправляє серверу пакет SYN із початковим відносним порядковим номером Seq = 0. Далі сервер повертає відповідь SYN, ACK, підтверджуючи отримання і встановлюючи Seq = 0, Ack = 1. Клієнт завершує рукостискання пакетом ACK (Seq = 1, Ack = 1), після чого канал готовий до роботи. Фактична передача даних починається майже миттєво з пакетом PSH, ACK (Len: 460), який несе в собі прикладний HTTP-запит клієнта.

Оскільки для сесій діє режим Persistent Connection (keep-alive), після виконання перших запитів розрив каналів не відбувається – вони залишаються активними для подальшого транзиту контенту.

Метод ковзного вікна



No.	Time	Source	Destination	Protocol	Length	Info
381	4.593740900	192.168.0.111	146.190.62.39	TCP	66	57444 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
357	4.774662800	192.168.0.111	146.190.62.39	TCP	66	51000 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
360	4.775905700	146.190.62.39	192.168.0.111	TCP	66	80 → 57444 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
361	4.776199000	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
367	4.779230200	192.168.0.111	146.190.62.39	HTTP	514	GET / HTTP/1.1
390	4.955999700	146.190.62.39	192.168.0.111	TCP	66	80 → 51000 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
391	4.956188000	192.168.0.111	146.190.62.39	TCP	54	51000 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
393	4.959259600	146.190.62.39	192.168.0.111	TCP	54	80 → 57444 [ACK] Seq=1 Ack=461 Win=64128 Len=0
394	4.960118900	146.190.62.39	192.168.0.111	TCP	1514	80 → 57444 [ACK] Seq=1 Ack=461 Win=64128 Len=1460 [TCP PDU reassembled in 395]
395	4.960118900	146.190.62.39	192.168.0.111	HTTP	1350	HTTP/1.1 200 OK (text/html)
396	4.960287300	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=461 Ack=2757 Win=65280 Len=0
419	5.107490600	192.168.0.111	146.190.62.39	HTTP	400	GET /js/init.min.js HTTP/1.1
522	5.294180000	146.190.62.39	192.168.0.111	TCP	1514	80 → 57444 [ACK] Seq=2757 Ack=807 Win=64128 Len=1460 [TCP PDU reassembled in 523]
523	5.294180000	146.190.62.39	192.168.0.111	HTTP	440	HTTP/1.1 200 OK (application/javascript)
526	5.294405900	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=807 Ack=4603 Win=65280 Len=0
527	5.324566400	192.168.0.111	146.190.62.39	HTTP	418	GET /css/style.min.css HTTP/1.1
528	5.325035500	192.168.0.111	146.190.62.39	HTTP	423	GET /css/style-wide.min.css HTTP/1.1
529	5.504668800	146.190.62.39	192.168.0.111	TCP	1514	80 → 57444 [ACK] Seq=4603 Ack=1171 Win=64128 Len=1460 [TCP PDU reassembled in 541]
530	5.504650800	146.190.62.39	192.168.0.111	TCP	2974	80 → 57444 [PSH, ACK] Seq=6063 Ack=1171 Win=64128 Len=2920 [TCP PDU reassembled in 541]
531	5.504732400	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=1171 Ack=8983 Win=65280 Len=0
532	5.505780800	146.190.62.39	192.168.0.111	TCP	5894	80 → 57444 [PSH, ACK] Seq=8983 Ack=1171 Win=64128 Len=5840 [TCP PDU reassembled in 541]
533	5.505891800	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=1171 Ack=14823 Win=65280 Len=0
534	5.506328600	146.190.62.39	192.168.0.111	TCP	4434	80 → 57444 [PSH, ACK] Seq=14823 Ack=1171 Win=64128 Len=4380 [TCP PDU reassembled in 541]
535	5.506455700	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=1171 Ack=19203 Win=65280 Len=0
536	5.506539700	146.190.62.39	192.168.0.111	TCP	54	80 → 51000 [ACK] Seq=1 Ack=370 Win=64128 Len=0
538	5.559915400	192.168.0.111	146.190.62.39	HTTP	972	HTTP/1.1 200 OK (text/css)
539	5.684425700	146.190.62.39	192.168.0.111	TCP	54	51000 → 80 [ACK] Seq=370 Ack=919 Win=64512 Len=0
540	5.685149200	146.190.62.39	192.168.0.111	TCP	1514	80 → 57444 [ACK] Seq=19203 Ack=1171 Win=64128 Len=1460 [TCP PDU reassembled in 541]
541	5.685149200	146.190.62.39	192.168.0.111	HTTP	1232	HTTP/1.1 200 OK (text/css)
542	5.685298800	192.168.0.111	146.190.62.39	TCP	54	57444 → 80 [ACK] Seq=1171 Ack=23301 Win=65280 Len=0
643	5.881459900	192.168.0.111	146.190.62.39	HTTP	485	GET /css/images/banner-.svg HTTP/1.1
644	5.882685500	192.168.0.111	146.190.62.39	HTTP	500	GET /css/images/header-mejor-on-light.svg HTTP/1.1
645	5.883645400	192.168.0.111	146.190.62.39	TCP	66	54359 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
678	5.944321100	192.168.0.111	146.190.62.39	TCP	66	50861 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
879	6.059325500	146.190.62.39	192.168.0.111	TCP	66	80 → 54359 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=128
880	6.059481300	192.168.0.111	146.190.62.39	TCP	54	54359 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Рисунок 18 – Список пакетів у WireShark

Для кращого розуміння принципу ковзного вікна краще досліджувати його на сторінці списку пакетів. Даний малюнок дозволяє наочно відстежити математику механізму ковзного вікна між клієнтом та сервером за трьома параметрами: Seq

(номер байта відправника), Ack (номер очікуваного байта від співрозмовника) та Win (розмір буфера).

1) Ініціалізація та масштабування вікна:

У стартовому пакеті № 301 [SYN] клієнт заявляє базове вікно $Win=65535$ та коефіцієнт масштабування $WS=256$. Сервер у пакеті № 360 [SYN, ACK] погоджується на ці умови, транслюючи свої параметри ($Win=64240$, $WS=128$).

2) Динаміка зсуву вікна через Seq та Ack:

Зсув меж ковзного вікна відбувається щоразу, коли сторона успішно приймає дані й підтверджує їх:

- Пакет № 367: Клієнт надсилає HTTP-запит GET / із параметрами $Seq=1$, $Ack=1$ та довжиною даних $Len=460$ байтів .
- Пакет № 393: Сервер прийняв запит і зсуває ліву межу свого вікна, відправляючи [ACK] з $Ack=461$ ($1 + 460$). Це означає, що перші 460 байтів отримано, очікуються дані з 461-го. Його власний Seq при цьому залишається 1.
- Пакети № 394 і № 395: Сервер видає тіло HTML-сторінки двома сегментами: $Len=1460$ (пакет № 394) та $Len=1296$ (пакет № 395). Обидва йдуть із позиції $Seq=1$ і сумарно несуть 2756 байтів.
- Пакет № 396: Клієнт успішно збирає весь HTML-код, додає обсяг до початкової позиції сервера ($1 + 2756$) і посуває своє вікно вперед, надсилаючи $Ack=2757$.

Власний порядковий номер клієнта тепер закономірно зміщується до $Seq=461.3$.

3) Керування потоком

Поле Win відображає поточний залишок вільного місця в системному буфері. Клієнт заявляє готовність приймати великі порції даних. Якби вебсервер надсилав пакети швидше, ніж браузер устигає їх обробляти, значення Win у пакетах від клієнта почало б стрімко падати. Побачивши це, ковзне вікно сервера автоматично стиснулося б, і він призупинив би відправку, захищаючи мережевий стик від переповнення пам'яті та втрати пакетів.

Аналіз TCP Stream Graph

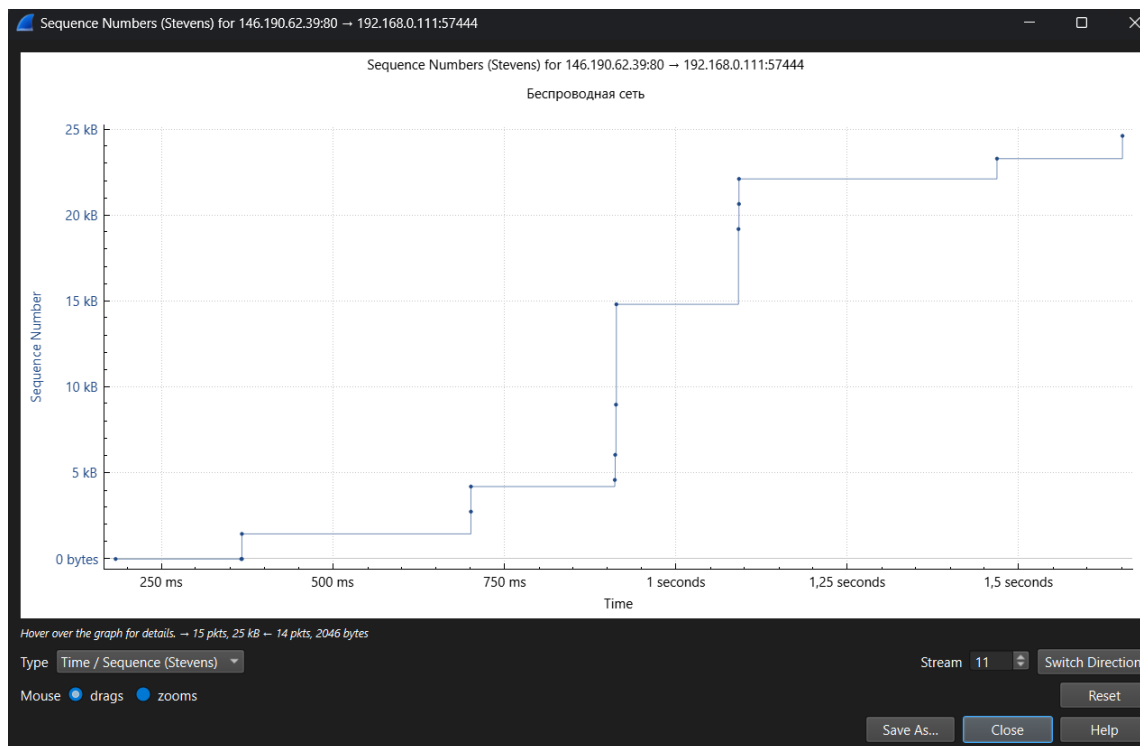


Рисунок 19 – Графік Time sequence (Stevens)

Цей графік відображає залежність порядкових номерів TCP від часу для конкретного потоку обміну даними. Він побудований для напрямку від віддаленого вебсервера до ПК. По осі абсцис відкладено час у мілісекундах від початку сесії, а по осі ординат – відносні порядкові номери байтів (Sequence Number), що дозволяє наочно оцінити динаміку надходження контенту та рівномірність завантаження файлів сайту.

Ступінчаста структура графіка чітко показує, що передача даних відбувалася не безперервним монотонним потоком, а дискретними порціями у моменти обробки HTTP-запитів. Горизонтальні плато на лінії графіку відображають періоди мережевого очікування, коли поточна порція даних уже завантажилась і комп'ютер розбирав отриманий код сторінки, перш ніж відправити запит на наступні елементи сайту.

Найпотужніші сплески завантаження зафіксовані в районі 900 мілісекунд та 1.1 секунди, де графік стрімко злітає вгору майже вертикально, проходячи кроки від 5 до 15 кілобайт і далі до 22 кілобайт. Ці зони відповідають передачі об'ємних прикладних

файлів. Рівна лінійна структура сходинок без розривів, просідань чи повторів точок свідчить про ідеальну стабільність бездротового каналу зв'язку.

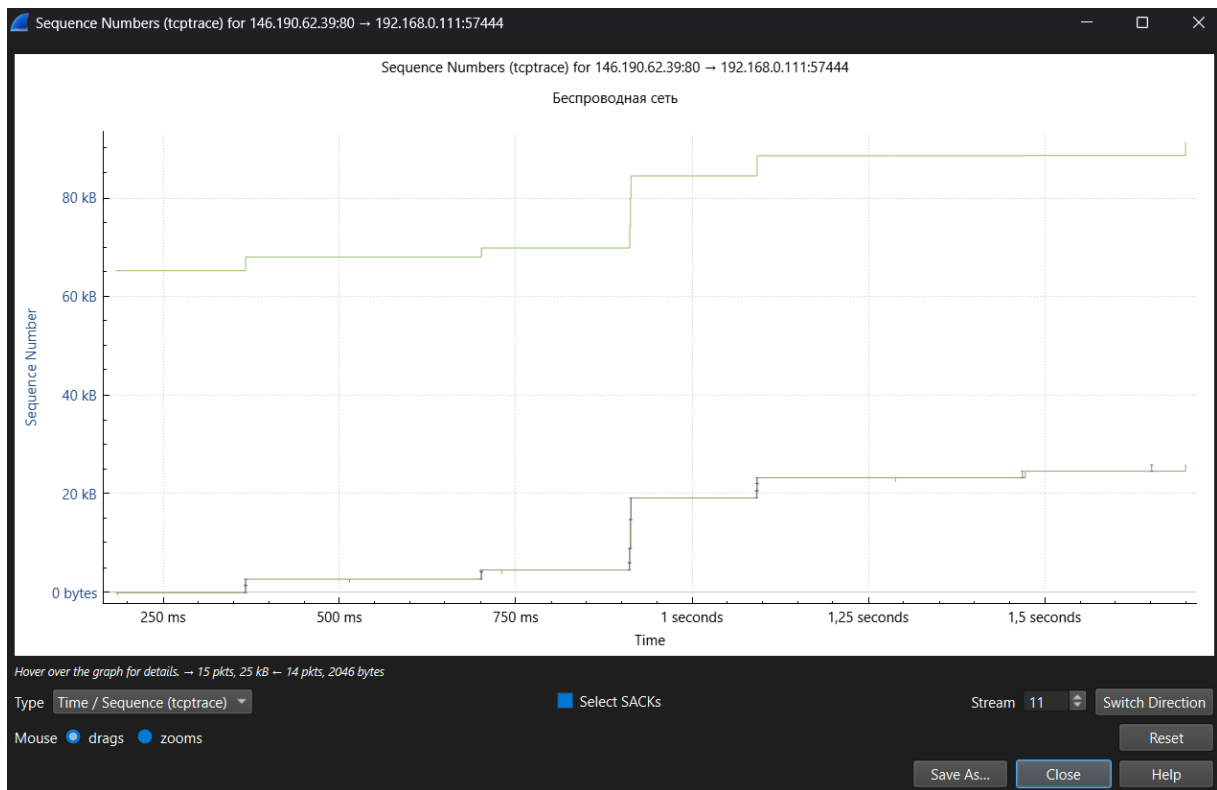


Рисунок 20 – Графік Time sequence (tcptrace)

Цей графік відображає часову залежність порядкових номерів TCP, побудовану за допомогою розширеного алгоритму tcptrace. На відміну від класичного графіка Стівенса, формат tcptrace одночасно візуалізує кілька критичних метрик транспортного рівня: нижня коричнева лінія показує фактично відправлені сервером порядкові номери, вертикальні сегменти над нею позначають межі та розмір окремих пакетів, а верхня зелена лінія відображає поточну межу вікна підтвердження.

Верхня зелена лінія повторює форму та вигини нижньої коричневої кривої, але проходить значно вище, на стабільній відстані близько 65 Кбайт. Це свідчить про те, що клієнт під час рукостискання та обміну задекларував великий і стабільний розмір приймального буфера, надаючи серверу величезний запас вільного місця для безперервного надсилання пакетів.

Стабільний паралельний рух кривих без перетинів, зближень чи коливань вказує на бездоганну якість мережевого з'єднання. Оскільки нижня лінія

відправлених даних ніде не наближається впритул до верхньої межі вікна, сервер жодного разу не зіткнувся зі станом заповнення буфера клієнта, який змусив би його штучно зупинити передачу. На графіку повністю відсутні петлі, накладання точок або просідання, що підтверджує повну відсутність втрат пакетів, дубльованих підтверджень чи повторних відправок.

ВИСНОВКИ

У ході виконання лабораторної роботи було успішно засвоєно практичні навички роботи з сучасним аналізатором мережевого трафіку Wireshark, за допомогою якого здійснено перехоплення, фільтрацію та комплексний аналіз пакетів у межах незашифрованої сесії прикладного протоколу HTTP. Сформована статистика розподілу трафіку наочно підтвердила ієрархічний принцип інкапсуляції даних: прикладні HTTP-повідомлення послідовно загортаються в транспортні сегменти TCP, мережеві пакети IPv4 та фізичні кадри Ethernet для безпомилкової доставки між кінцевими вузлами мережі. Дослідження фонового мультикаст-трафіку та широкомовних запитів протоколу ARP дозволило закріпити розуміння механізмів динамічного відображення логічних IP-адрес у фізичні MAC-адреси пристроїв у межах одного локального сегмента підмережі.

На основі детального дослідження часових діаграм взаємодії було проаналізовано стратегії оптимізації сучасними браузером. Встановлено, що для прискорення завантаження вебсторінки клієнт ефективно використовує механізми паралельного відкриття кількох TCP-сокетів, що дозволяє одночасно викачувати базовий код сторінки, каскадні таблиці стилів, файли скриптів та графічні елементи інтерфейсу. Проведена оцінка часових інтервалів показала високу швидкість обробки клієнтських запитів віддаленим сервером, а аналіз ступеня використання смуги пропускання підтвердив, що короткочасний текстовий HTTP-обмін створює мінімальне питоме навантаження на сучасні високошвидкісні канали зв'язку.

Особливу увагу в роботі було приділено дослідженню фундаментальних алгоритмів забезпечення надійності транспортного рівня TCP за допомогою побудови та аналізу графіків Стівенса та утиліти tcptrace. Експериментально підтверджено повний життєвий цикл TCP-з'єднання, починаючи від процедури тристороннього рукошлякування для синхронізації початкових параметрів і закінчуючи асинхронним обміном даними з прапорцями PSH, ACK. Наочна візуалізація метрик довела високу ефективність методу ковзного вікна гарантує доставку даних без втрат, а безперервний моніторинг параметра Window реалізує механізм зворотного зв'язку,

повністю виключаючи ризик переповнення буфера приймача клієнта та оптимізуючи швидкість передачі під поточний стан бездротового каналу зв'язку.